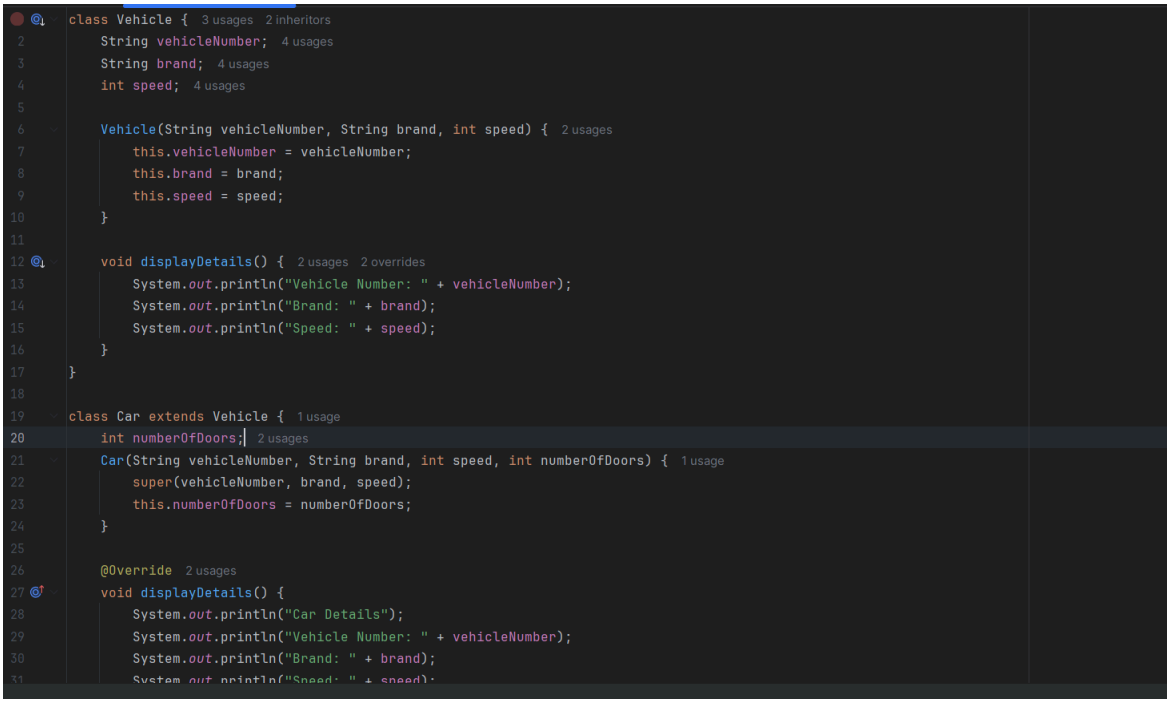
Tutorial 6 (31/8/2026)

Name: V.I. Sahaana
RegNo: RA2511033010078
Section: AM2

1. A vehicle rental company manages Cars and Bikes. Create a Java program with a parent class Vehicle containing vehicleNumber, brand, speed, and displayDetails(). Derive Car and Bike from Vehicle, adding numberOfDoors and hasGear respectively. Override displayDetails() in both subclasses and demonstrate runtime polymorphism by using a Vehicle reference to invoke the appropriate overridden method.

Ans:



```
class Vehicle {  3 usages  2 inheritors
    String vehicleNumber;  4 usages
    String brand;  4 usages
    int speed;  4 usages

    Vehicle(String vehicleNumber, String brand, int speed) {  2 usages
        this.vehicleNumber = vehicleNumber;
        this.brand = brand;
        this.speed = speed;
    }

    void displayDetails() {  2 usages  2 overrides
        System.out.println("Vehicle Number: " + vehicleNumber);
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed);
    }
}

class Car extends Vehicle {  1 usage
    int numberOfDoors;  2 usages
    Car(String vehicleNumber, String brand, int speed, int numberOfDoors) {  1 usage
        super(vehicleNumber, brand, speed);
        this.numberOfDoors = numberOfDoors;
    }

    @Override  2 usages
    void displayDetails() {
        System.out.println("Car Details");
        System.out.println("Vehicle Number: " + vehicleNumber);
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed);
```

```java
class Vehicle {  3 usages  2 inheritors
    void displayDetails() {  2 usages  2 overrides
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed);
    }
}

class Car extends Vehicle {  1 usage
    int numberOfDoors;  2 usages
    Car(String vehicleNumber, String brand, int speed, int numberOfDoors) {  1 usage
        super(vehicleNumber, brand, speed);
        this.numberOfDoors = numberOfDoors;
    }

    @Override  2 usages
    void displayDetails() {
        System.out.println("Car Details");
        System.out.println("Vehicle Number: " + vehicleNumber);
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed);
        System.out.println("Number of Doors: " + numberOfDoors);
    }
}
class Bike extends Vehicle {  1 usage
    boolean hasGear;  2 usages

    Bike(String vehicleNumber, String brand, int speed, boolean hasGear) {  1 usage
        super(vehicleNumber, brand, speed);
        this.hasGear = hasGear;
    }
    @Override  2 usages
```

```java
class Bike extends Vehicle {  1 usage
        this.hasGear = hasGear;
    }
    @Override  2 usages
    void displayDetails() {
        System.out.println("Bike Details");
        System.out.println("Vehicle Number: " + vehicleNumber);
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed);
        System.out.println("Has Gear: " + hasGear);
    }
}
public class VehicleDemo {
    public static void main(String[] args) {

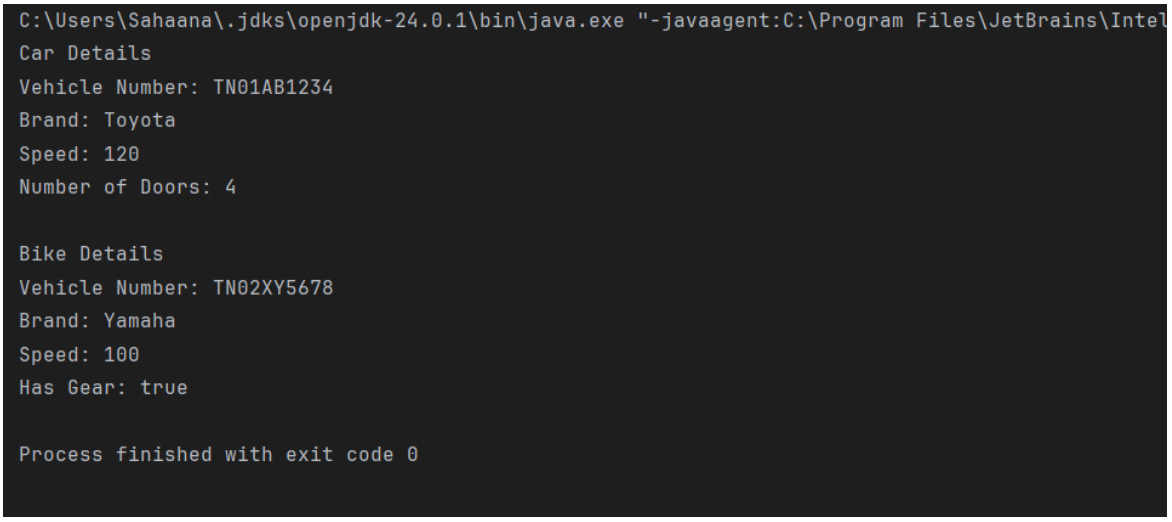
        Vehicle vehicle;

        vehicle = new Car( vehicleNumber: "TN01AB1234",  brand: "Toyota",  speed: 120,  numberOfDoors: 4);
        vehicle.displayDetails();

        System.out.println();

        vehicle = new Bike( vehicleNumber: "TN02XY5678",  brand: "Yamaha",  speed: 100,  hasGear: true);
        vehicle.displayDetails();
    }
}
```

Output:


C:\Users\Sahaana\.jdks\openjdk-24.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\Intel
Car Details
Vehicle Number: TN01AB1234
Brand: Toyota
Speed: 120
Number of Doors: 4
Bike Details
Vehicle Number: TN02XY5678
Brand: Yamaha
Speed: 100
Has Gear: true
Process finished with exit code 0

2. An online shopping application supports Credit Card, UPI, and Net Banking payments. Create a Java program by declaring a Payment interface with a pay(double amount) method. Implement the interface in CreditCardPayment, UPIPayment, and NetBankingPayment classes with different implementations. Use a Payment reference to invoke pay() for different payment methods.

Ans:

```java
interface Payment {  4 usages  3 implementations
    void pay(double amount);  3 usages  3 implementations
}
class CreditCardPayment implements Payment {  1 usage

    public void pay(double amount) {  3 usages
        System.out.println("Paid Rs." + amount + " using Credit Card");
    }
}
class UPIPayment implements Payment {  1 usage

    public void pay(double amount) {  3 usages
        System.out.println("Paid Rs." + amount + " using UPI");
    }
}
class NetBankingPayment implements Payment {  1 usage

    public void pay(double amount) {  3 usages
        System.out.println("Paid Rs." + amount + " using Net Banking");
    }
}
public class PaymentDemo {
    public static void main(String[] args) {

        Payment payment;

        payment = new CreditCardPayment();
        payment.pay( amount: 5000);

        payment = new UPIPayment();
```

```java
        payment = new UPIPayment();
        payment.pay( amount: 2500);

        payment = new NetBankingPayment();
        payment.pay( amount: 7500);
    }
}
```

Output:

```
C:\Users\Sahaana\.jdks\openjdk-24.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Communi
Paid Rs.5000.0 using Credit Card
Paid Rs.2500.0 using UPI
Paid Rs.7500.0 using Net Banking

Process finished with exit code 0
```

3. A mobile banking application provides basic login and logout features. An advanced version additionally supports fingerprint and face-recognition authentication. Create a Java program with a BasicSecurity interface and an AdvancedSecurity interface that extends it. Implement AdvancedSecurity in a BankingApplication class and demonstrate all the security operations.

Ans:

```java
interface BasicSecurity {  1 usage  2 implementations
    void login();  1 usage  1 implementation
    void logout();  1 usage  1 implementation
}

interface AdvancedSecurity extends BasicSecurity {  1 usage  1 implementation
    void fingerprintAuthentication();  1 usage  1 implementation
    void faceAuthentication();  1 usage  1 implementation
}

class BankingApplication implements AdvancedSecurity {  2 usages

    public void login() {  1 usage
        System.out.println("Login successful");
    }

    public void logout() {  1 usage
        System.out.println("Logout successful");
    }

    public void fingerprintAuthentication() {  1 usage
        System.out.println("Fingerprint authentication successful");
    }

    public void faceAuthentication() {  1 usage
        System.out.println("Face recognition successful");
    }
}
```

```java
public class SecurityDemo {
    public static void main(String[] args) {

        BankingApplication bank = new BankingApplication();

        bank.login();
        bank.fingerprintAuthentication();
        bank.faceAuthentication();
        bank.logout();
    }
}
```

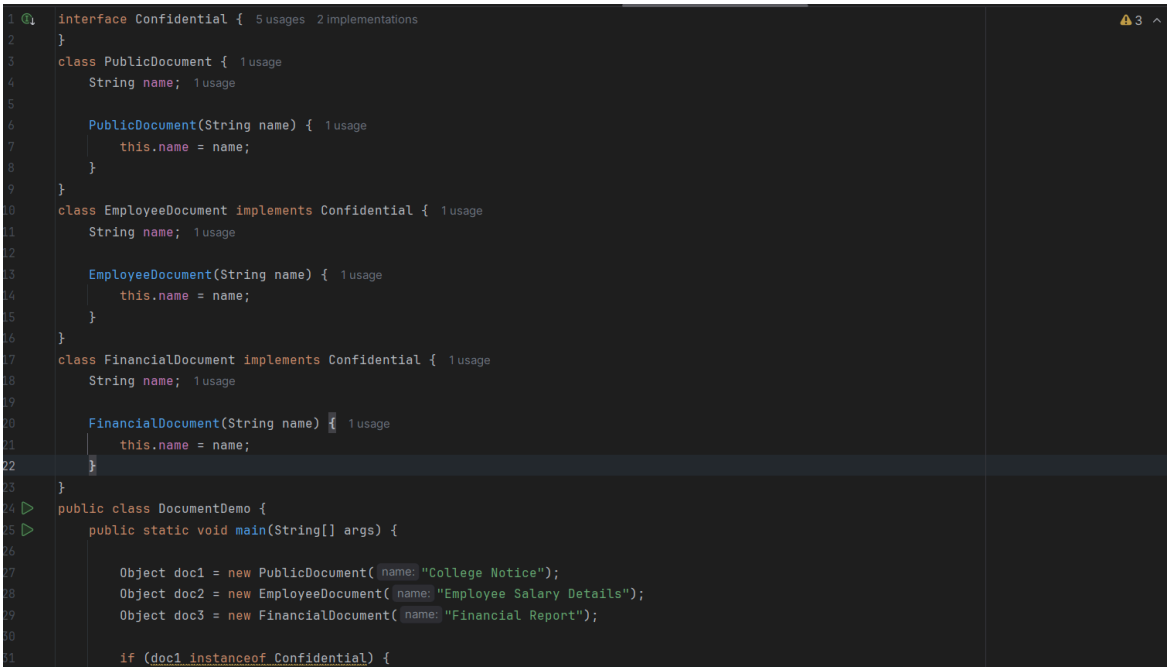
Output:

```
C:\Users\Sahaana\.jdks\openjdk-24.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.3\lib
Login successful
Fingerprint authentication successful
Face recognition successful
Logout successful

Process finished with exit code 0
```

4. A document management system stores different types of documents. Create a tagging interface Confidential and implement it in selected document classes. Use instanceof to identify whether a document is confidential.

Ans:



```
interface Confidential {  5 usages  2 implementations
}
class PublicDocument {  1 usage
    String name;  1 usage

    PublicDocument(String name) {  1 usage
        this.name = name;
    }
}
class EmployeeDocument implements Confidential {  1 usage
    String name;  1 usage

    EmployeeDocument(String name) {  1 usage
        this.name = name;
    }
}
class FinancialDocument implements Confidential {  1 usage
    String name;  1 usage

    FinancialDocument(String name) {  1 usage
        this.name = name;
    }
}
public class DocumentDemo {
    public static void main(String[] args) {

        Object doc1 = new PublicDocument( name: "College Notice");
        Object doc2 = new EmployeeDocument( name: "Employee Salary Details");
        Object doc3 = new FinancialDocument( name: "Financial Report");

        if (doc1 instanceof Confidential) {
```

```java
public class DocumentDemo {
    public static void main(String[] args) {

        Object doc1 = new PublicDocument( name: "College Notice");
        Object doc2 = new EmployeeDocument( name: "Employee Salary Details");
        Object doc3 = new FinancialDocument( name: "Financial Report");

        if (doc1 instanceof Confidential) {
            System.out.println("College Notice is Confidential");
        } else {
            System.out.println("College Notice is not Confidential");
        }

        if (doc2 instanceof Confidential) {
            System.out.println("Employee Salary Details is Confidential");
        } else {
            System.out.println("Employee Salary Details is not Confidential");
        }

        if (doc3 instanceof Confidential) {
            System.out.println("Financial Report is Confidential");
        } else {
            System.out.println("Financial Report is not Confidential");
        }
    }
}
```

Output:

```
C:\Users\Sahaana\.jdks\openjdk-24.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.3
College Notice is not Confidential
Employee Salary Details is Confidential
Financial Report is Confidential

Process finished with exit code 0
```

5. Develop a Java program for a college management system by creating separate packages for Student and Course management. Create a Student class with student details and a Course class with course details. Place each class in its respective package, import them into a Main class, and display the student and course information.

Ans:

```
package Student;

public class Student {  no usages

    String name;  2 usages
    int rollNo;  2 usages

    public Student(String name, int rollNo) {  no usages
        this.name = name;
        this.rollNo = rollNo;
    }

    public void displayStudent() {  no usages
        System.out.println("Student Name: " + name);
        System.out.println("Roll Number: " + rollNo);
    }
}
```

```java
package course;

public class Course {  no usages

    String courseName;  2 usages
    int courseCode;  2 usages

    public Course(String courseName, int courseCode) {  no usages
        this.courseName = courseName;
        this.courseCode = courseCode;
    }

    public void displayCourse() {  no usages
        System.out.println("Course Name: " + courseName);
        System.out.println("Course Code: " + courseCode);
    }
}
```

```java
import Student.Student;
import course.Course;

public class CollegeDemo {

    public static void main(String[] args) {

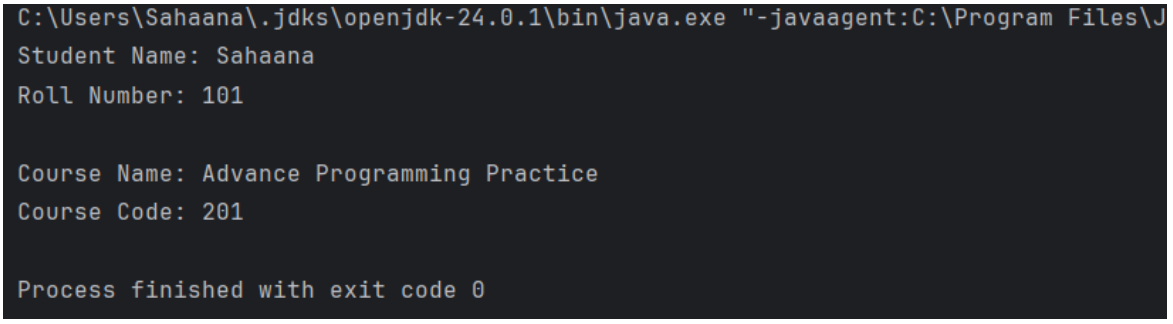
        Student s = new Student( name: "Sahaana",  rollNo: 101);
        Course c = new Course( courseName: "Advance Programming Practice",  courseCode: 201);

        s.displayStudent();

        System.out.println();

        c.displayCourse();
    }
}
```

Output:



6. Develop a Java-based Online Banking System that supports different types of accounts and payment methods. Create a parent class Account and derive SavingsAccount and CurrentAccount from it. Override the displayDetails() method in both subclasses and demonstrate runtime polymorphism using an Account reference.

   Create a Payment interface with a pay(double amount) method and implement it in UPIPayment and CardPayment. Create another interface SecurePayment that extends Payment and adds a verifyPayment() method. Implement it in UPIPayment.

Create a tagging interface OnlineTransaction and use it to identify online payment classes. Organize the classes into suitable packages and create a Main class to demonstrate the complete system.

Ans:

```
package accounts;

public class Account {  no usages

    protected int accountNumber;  2 usages
    protected String holderName;  2 usages
    protected double balance;  2 usages

    public Account(int accountNumber, String holderName, double balance) {  no usages
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    public void displayDetails() {  no usages
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Holder Name: " + holderName);
        System.out.println("Balance: " + balance);
    }
}
```

```java
package accounts;

public class SavingsAccount extends Account {  2 usages

    double interestRate;  2 usages

    public SavingsAccount(int accountNumber, String holderName,  1 usage
                          double balance, double interestRate) {

        super(accountNumber, holderName, balance);
        this.interestRate = interestRate;
    }

    @Override  2 usages
    public void displayDetails() {
        System.out.println("Savings Account");
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Holder Name: " + holderName);
        System.out.println("Balance: " + balance);
        System.out.println("Interest Rate: " + interestRate + "%");
    }
}
```

```java
package accounts;

public class CurrentAccount extends Account {  2 usages

    double overdraftLimit;  2 usages

    public CurrentAccount(int accountNumber, String holderName,  1 usage
                          double balance, double overdraftLimit) {

        super(accountNumber, holderName, balance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override  2 usages
    public void displayDetails() {
        System.out.println("Current Account");
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Holder Name: " + holderName);
        System.out.println("Balance: " + balance);
        System.out.println("Overdraft Limit: " + overdraftLimit);
    }
}
```

```java
package payments;

public interface Payment {  4 usages  3 implementations

    void pay(double amount);  3 usages  2 implementations
}
```

```java
package payments;

public interface SecurePayment extends Payment {  1 usage

    void verifyPayment();  1 usage  1 implementation
}
```

```java
package payments;

import transaction.OnlineTransaction;

public class UPIPayments implements SecurePayment, OnlineTransaction {  5 usages

    public void pay(double amount) {  3 usages
        System.out.println("Paid Rs." + amount + " using UPI");
    }

    public void verifyPayment() {  1 usage
        System.out.println("UPI payment verified");
    }
}
```

```java
package payments;

import transaction.OnlineTransaction;

public class CardPayment implements Payment, OnlineTransaction {  3 usages

    public void pay(double amount) {  3 usages
        System.out.println("Paid Rs." + amount + " using Card");
    }
}
```

```java
package transaction;

public interface OnlineTransaction {  8 usages  2 implementations
}
```

```java
import accounts.Account;
import accounts.SavingsAccount;
import accounts.CurrentAccount;

import payments.Payment;
import payments.UPIPayments;
import payments.CardPayment;

import transaction.OnlineTransaction;

public class BankingDemo {

    public static void main(String[] args) {

        Account account;

        account = new SavingsAccount( accountNumber: 101,  holderName: "Sahaana",  balance: 50000,  interestRate: 5.5);
        account.displayDetails();

        System.out.println();

        account = new CurrentAccount( accountNumber: 102,  holderName: "Rahul",  balance: 75000,  overdraftLimit: 20000);
        account.displayDetails();

        System.out.println();

        Payment payment;

        payment = new UPIPayments();
        payment.pay( amount: 2500);
```

```java
    public static void main(String[] args) {
        payment.pay( amount: 2500);

        System.out.println();

        payment = new CardPayment();
        payment.pay( amount: 5000);

        System.out.println();

        UPIPayments upi = new UPIPayments();

        upi.pay( amount: 3000);
        upi.verifyPayment();

        System.out.println();

        OnlineTransaction transaction;

        transaction = new UPIPayments();

        if (transaction instanceof OnlineTransaction) {
            System.out.println("UPI is an online transaction");
        }

        transaction = new CardPayment();

```

```java
        if (transaction instanceof OnlineTransaction) {
            System.out.println("Card is an online transaction");
        }
    }
}
```

Output:

```
C:\Users\Sahaana\.jdks\openjdk-24.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.3\lib\idea_rt
Savings Account
Account Number: 101
Holder Name: Sahaana
Balance: 50000.0
Interest Rate: 5.5%

Current Account
Account Number: 102
Holder Name: Rahul
Balance: 75000.0
Overdraft Limit: 20000.0

Paid Rs.2500.0 using UPI
```